

Machine Learning

— field notes for the curious —

Foundations

Algorithms

Evaluation

Real Projects

Interview-Ready

01-05

Foundations, data &
prep

06-09

Algorithms & features

10-14

Tuning, ensembles &
recap

What is Machine Learning?

The one-line version: give a computer examples instead of instructions, and let it find the pattern.

AI → ML → DL

Artificial Intelligence is the broad goal of building machines that act intelligently. Machine Learning is one way to get there: systems that improve at a task by learning from data instead of following hand-written rules. Deep Learning is a specialised branch of ML that uses layered neural networks to learn very complex patterns, usually from large datasets.

What ML Actually Does

Instead of writing explicit rules, we feed a model examples. It searches for a pattern that connects inputs to outputs, then uses that pattern to make predictions on data it has never seen before. The whole point is **generalisation**, not memorisation.

Rules vs. Learning

	Classic Programming	Machine Learning
Input	Data + Rules	Data + Answers
Output	Result	Rules (the model)
Best when	Logic is known	Logic is too complex to hand-code

The Learning Loop

1. Collect data → 2. Clean & prepare it → 3. Train a model → 4. Check it on data it hasn't seen → 5. Ship predictions. Every real ML project cycles through this loop more than once.

Why it matters: the field spans fraud detection, medical imaging, recommendation engines, self-driving perception, and spam filters — anywhere the "rules" are too messy for a human to write down directly.

Every ML problem falls into one of four learning styles — the data you have decides which one fits.

Have labels?	Have a reward signal?	Go with
Yes, plenty	—	Supervised
No	No	Unsupervised
Few	No	Semi-Supervised
—	Yes	Reinforcement

Data, the Raw Material

Before any model can learn, you need to understand the shape and structure of what you're feeding it.

The Vocabulary

- **Dataset** — the full collection of records.
- **Sample / row** — one record.
- **Feature (X)** — an input variable describing the sample.
- **Label / target (y)** — the value we want to predict.

Numeric vs. Categorical

Numeric: discrete (countable, e.g. number of items) or continuous (measured, e.g. temperature).

Categorical: nominal (no order — colour, city) or ordinal (has order — low/medium/high).

Train / Validation / Test

~70%

Training — the model learns patterns here.

~15%

Validation — tune settings, compare models.

~15%

Test — touched once, for an honest final score.

Why Split at All?

- Stops the model from just memorising the training set.
- Lets you tune fairly, without peeking at the final exam.
- Gives an unbiased read on real-world performance.

Rule #1: No Leakage

Never let information from validation or test data influence training — not even indirectly through scaling or feature selection done before the split. Leakage is the single most common reason a model looks great in notebooks and fails in production.

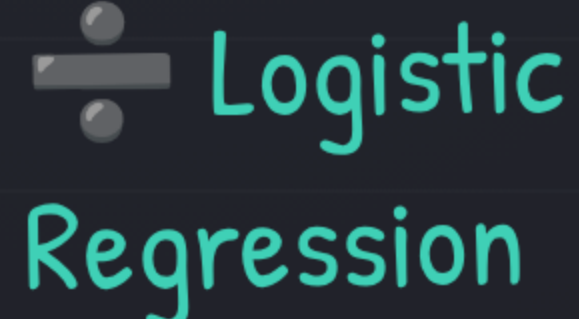
Raw data is rarely model-ready — preprocessing turns it into something an algorithm can actually learn from.

Look before you model. EDA is where you build intuition for what the data is actually telling you.

Don't just show a chart — say what it means for the model. "Right-skewed income, so I log-transformed it before regression" is a much stronger answer than the chart alone.

Regression predicts a number. Start with the simplest line that fits — add complexity only when the data demands it.

Classification predicts a category. Different algorithms draw the decision boundary in very different ways.



Classifies a point by the majority vote of its closest K neighbours. No real "training" — just distance lookups.



Applies Bayes' theorem assuming features are independent. Punches above its weight on text and spam tasks.



Situation

Reach for

KNN

Random Forest / Gradient Boosting

Naive Bayes or SVM

No labels, no answer key — just the shape hiding inside the data.

Needs K chosen in advance; assumes roughly round, similarly-sized clusters.

Builds a tree of nested clusters — bottom-up (merge closest pairs) or top-down (keep splitting). Visualised with a dendrogram, and you don't need to fix K upfront.

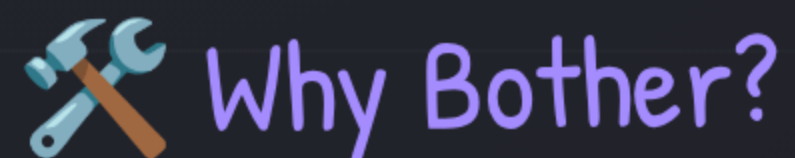
Groups points that are densely packed together and labels sparse points as noise. Handles odd-shaped clusters and finds outliers as a side effect — no K required.

Finding the rare points that don't fit the normal pattern — fraud, faults, intrusions. Isolation Forest, One-Class SVM, and DBSCAN's own "noise" label are common tools.

Quick Comparison

Method	Needs K?	Cluster shape	Good for
K-Means	Yes	Round, similar size	Large datasets, fast
Hierarchical	No	Any	Small data, exploration
DBSCAN	No	Arbitrary	Noise & outlier detection

Better features, not just better models — this is often where the real accuracy gains come from.



A well-built feature can do more for accuracy than switching algorithms. Good features turn domain knowledge into signal the model can actually use.



- **Create** — combine columns (price $\times$ qty = total)
- **Select** — drop noisy or redundant features
- **Transform** — log/power transforms for skewed data
- **Encode** — turn categories into numbers
- **Scale** — put numeric features on the same range



Turning a continuous value like age into buckets (young / adult / senior) can expose non-linear patterns and tame outliers.



Sometimes two features together carry more signal than either alone — income $\times$ spending can reveal a pattern neither column shows by itself.

Watch out: building features from the test set, or from information that wouldn't exist at prediction time, is a sneaky form of data leakage.

A model is only as good as the metric you judge it by — and the wrong metric can hide a broken model.

Confusion Matrix

Classification Metrics

$$F1 = 2 \cdot (P \cdot R) / (P + R)$$



ROC-AUC

Regression Metrics

RMSE punishes big misses harder than MAE — pick based on whether large errors matter more.

A single lucky train/test split can be misleading — cross-validation and careful tuning make results you can trust.

Split the data into K parts, train on K-1 and validate on the last, rotate through all K, then average the scores. Far more reliable than a single train/validation split.

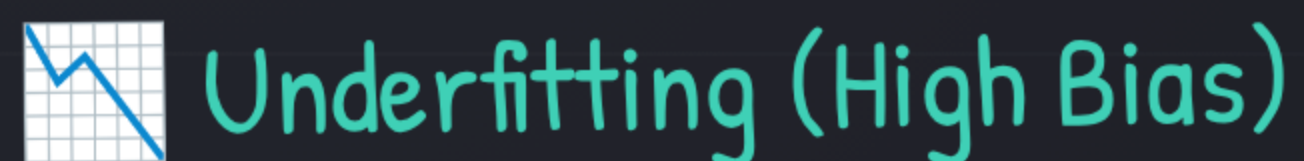
Keeps the same class ratio in every fold — essential when classes are imbalanced, so no fold accidentally ends up with almost none of the minority class.

Parameters are learned by the model during training (weights, coefficients).
Hyperparameters are set by you beforehand (learning rate, tree depth, K).

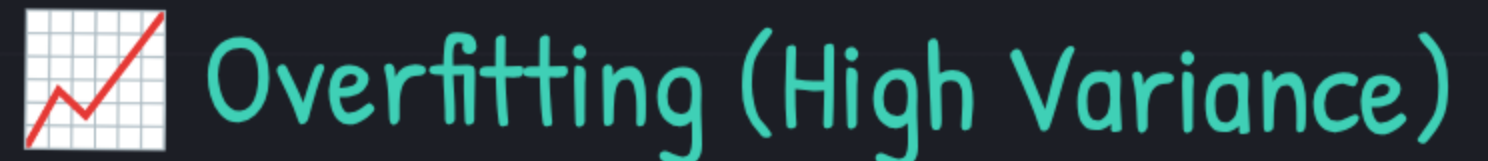
Grid search tries every combination — thorough but slow to scale. **Random search** samples combinations — usually finds a near-optimal setting much faster on large search spaces.

Split data → cross-validate on training folds → search hyperparameters → retrain the best config on all training data → evaluate once, on the untouched test set.

The eternal trade-off: too simple and it misses the pattern, too complex and it just memorises the data.



A line graph on a grid showing an upward trend. The line starts at a low point, rises to a peak, dips slightly, and then rises more steeply to its highest point.



The Trade-off



Fixing Overfitting

- 

Fixing Underfitting

- Increase model complexity
- Add more informative features
- Reduce over-strong assumptions
- Train for longer

Combine many decent models and you usually beat any single great one.

A single model's errors are noisy; averaging several models cancels out a lot of that noise. That's why ensembles — especially gradient boosting — dominate tabular-data leaderboards.

One page to hold the whole journey together before an interview or a review.

Problem	Try first
Regression	Linear → Ridge/Lasso → Boosting
Classification	Logistic → Random Forest → Boosting
Clustering	K-Means → DBSCAN

- Supervised vs. unsupervised — with an example
- Bias-variance trade-off, in your own words
- Precision vs. recall — when does each matter?
- What is data leakage, and how do you prevent it?
- How would you explain a project end-to-end?
- Why isn't accuracy always the right metric?

Cheat Sheet 14/14